DATABASE MANAGEMENT SYSTEMS

VEHICLE SERVICE CENTRE

Management Database Using PostgreSQL and SQL

Case Study	149 – Vehicle Service Centre
Project Type	Relational database design and SQL query implementation
Database Platform	PostgreSQL (executed through VS Code / SQL client)
Core Concepts	Primary Keys, Foreign Keys, JOIN, Aggregate Functions, GROUP BY, HAVING

Prepared by: Antara Palwankar

B.Tech Computer Science and Engineering

PROJECT DOCUMENTATION

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Aim and Objectives
5. Scope of the Project
6. Technologies and Tools Used
7. Fundamental DBMS Concepts
8. Database Design and ER Model
9. Table Structure and Data Dictionary
10. SQL Implementation
11. Explanation of Required Queries
12. Sample Data and Expected Results
13. Testing and Validation
14. Advantages and Limitations
15. Future Enhancements
16. Conclusion
17. Viva-Voce Questions and Answers

Appendix A: Complete SQL Script

Note: This document describes the three-table implementation developed for the case study. Sample records and expected query results are illustrative and can differ if the database contains different data.

1. Abstract

The Vehicle Service Centre Management Database is a beginner-level relational database project developed for Case Study 149. Its purpose is to store and organize information about customers, their vehicles, and the service jobs performed on those vehicles. The project uses PostgreSQL and SQL to create tables, insert records, connect related data, and retrieve useful information for service-centre operations.

The database contains three main tables: **customers**, **vehicles**, and **service_jobs**. Each table has a primary key to identify its records. Foreign keys connect vehicles to customers and service jobs to vehicles. These relationships allow the service centre to trace a service record back to the vehicle and its owner.

The SQL queries demonstrate the retrieval of vehicle service history, calculation of total service revenue, calculation of average service cost per vehicle, identification of customers whose spending exceeds a chosen amount, and identification of vehicles with more than two service records. The implementation focuses on fundamental SQL concepts rather than advanced query techniques.

2. Introduction

A vehicle service centre handles customers, vehicles, and maintenance or repair work. A single customer may own multiple vehicles, and each vehicle may visit the service centre several times. Every visit can have a different service date, description, and cost. If this information is kept only in separate paper files or repeated in one large spreadsheet, it may become difficult to search, update, and analyze.

A database provides a structured way to store this information. In a relational database, information is organized into tables made up of rows and columns. Related tables can be connected through keys. SQL (Structured Query Language) is used to define the tables, insert records, retrieve information, and calculate summaries.

This project models a small service centre using three related tables. The design separates customer details from vehicle details and service records. This makes it easier to maintain the data and answer practical questions such as: Who owns this vehicle? What services were performed? How much revenue was recorded? Which vehicles have multiple service visits?

3. Problem Statement

The service centre needs a simple database to manage customer details, vehicle details, and service-job records. It should be possible to associate each vehicle with its owner and each service job with the relevant vehicle. The database should also support basic analysis of service costs and service frequency.

The required outputs include a vehicle-owner service-history report, total service revenue, average service cost by vehicle, customers whose total recorded spending is greater than ■5,000, and vehicles with more than two service records. These requirements are implemented using primary keys, foreign keys, joins, aggregate functions, GROUP BY, and HAVING.

4. Aim and Objectives

Aim: To design and implement a relational database for a vehicle service centre using PostgreSQL and basic SQL.

Objectives:

- Store customer names and cities.
- Store vehicle registration numbers and models.
- Link each vehicle to its customer using a foreign key.
- Store each service date, description, and cost.
- Link each service job to a vehicle using a foreign key.
- Display vehicle owners with their service history.

• Calculate total service revenue using SUM().
• Calculate average service cost per vehicle using AVG().
• Identify customers spending more than ■5,000 using SUM(), GROUP BY, and HAVING.
• Identify vehicles with more than two service records using COUNT(), GROUP BY, and HAVING.

5. Scope of the Project

The current project covers the core database functions required by the case study. It creates and populates three tables, maintains key-based relationships, and runs SQL queries to retrieve and summarize data. It is intended as an academic database project and demonstration of relational design.

Included: customer, vehicle, and service-job records; primary and foreign keys; unique vehicle registration numbers; basic data types and constraints; INSERT and SELECT statements; JOIN operations; SUM, AVG, COUNT, MAX, and MIN; GROUP BY and HAVING.

Not included: a graphical booking application, employee and mechanic management, spare-parts inventory, invoices, payments, user authentication, automatic reminders, or online deployment. These can be added later if the project requirements expand.

6. Technologies and Tools Used

PostgreSQL: A relational database management system (RDBMS) used to create and manage the database. It supports SQL, constraints, and relational links between tables.

SQL: The language used to create tables, insert data, select records, join related tables, and calculate totals and averages.

Visual Studio Code: The editor used to prepare and organize the SQL script. SQL can be executed through a suitable PostgreSQL extension or through the PostgreSQL command-line client.

PostgreSQL client / psql: A client is needed to connect to a PostgreSQL server and execute the SQL statements. VS Code itself is an editor; the database server performs the actual database operations.

7. Fundamental DBMS Concepts

7.1 Data, Database, and DBMS

Data is raw information such as a customer name, registration number, service date, or cost. A **database** is an organized collection of related data. A **Database Management System (DBMS)** is software used to define, store, retrieve, update, and manage data. PostgreSQL is the DBMS used in this project.

7.2 Relational Database and Tables

A relational database stores information in tables. A table has columns (attributes) and rows (records). In this project, customers, vehicles, and service_jobs are separate tables because they represent different types of information.

7.3 Primary Key (PK)

A primary key uniquely identifies each row in a table. Its values must be unique and cannot be NULL. The project uses customer_id in customers, vehicle_id in vehicles, and service_id in service_jobs as primary keys. The SERIAL type in PostgreSQL generates integer values automatically for new records when the ID is omitted from an INSERT statement.

7.4 Foreign Key (FK)

A foreign key is a column that references a key in another table and helps maintain referential integrity. vehicles.customer_id references customers.customer_id. service_jobs.vehicle_id references vehicles.vehicle_id. A service record therefore cannot reference a vehicle ID that does not exist, and a vehicle cannot reference a customer ID that does not exist.

7.5 One-to-Many Relationship

One customer can own multiple vehicles, while each vehicle record refers to one customer. One vehicle can have multiple service-job records, while each service job refers to one vehicle. These are one-to-many relationships.

7.6 Constraints

NOT NULL requires a value to be supplied. **UNIQUE** prevents duplicate values in a column. **PRIMARY KEY** combines uniqueness and non-null identification for the key. **FOREIGN KEY** checks that the referenced parent record exists. The service_cost column uses DECIMAL(10,2) to store monetary values with two decimal places.

7.7 SQL Commands Used

CREATE TABLE defines a table. **INSERT INTO** adds records. **SELECT** retrieves records. **JOIN** combines related records across tables. **SUM()** totals numeric values, **AVG()** calculates an average, **COUNT()** counts records, **MAX()** returns the highest value, and **MIN()** returns the lowest value. **GROUP BY** forms groups for calculations, and **HAVING** filters groups based on an aggregate condition.

8. Database Design and ER Model

The database has three entities: CUSTOMER, VEHICLE, and SERVICE_JOB. The CUSTOMER entity stores customer information. The VEHICLE entity stores vehicle details and includes a foreign key to identify the owner. The SERVICE_JOB entity stores individual service events and includes a foreign key to identify the vehicle that received the service.

Relationship 1: CUSTOMER (1) — (Many) VEHICLE. A customer can own several vehicles. Each vehicle record is linked to one customer through customer_id.

Relationship 2: VEHICLE (1) — (Many) SERVICE_JOB. A vehicle can have several service records. Each service job is linked to one vehicle through vehicle_id.

Text representation of the ER model:

```
CUSTOMERS
---------
PK customer_id
   customer_name
   city
      |
      | 1 to many
      v
VEHICLES
--------
PK vehicle_id
   registration_no (UNIQUE)
   vehicle_model
FK customer_id -> CUSTOMERS.customer_id
      |
      | 1 to many
      v
SERVICE_JOBS
------------
PK service_id
FK vehicle_id -> VEHICLES.vehicle_id
   service_date
   service_description
   service_cost
```

PK means Primary Key; FK means Foreign Key. The arrows show the direction of the references from child tables to parent tables. The model allows the complete history of a vehicle to be retrieved by following the links from customers to vehicles to service jobs.

9. Table Structure and Data Dictionary

9.1 customers

Column	Data type	Key / constraint	Purpose
customer_id	SERIAL	Primary Key	Automatically generated identifier for each customer.
customer_name	VARCHAR(100)	NOT NULL	Stores the customer's name.
city	VARCHAR(50)	NOT NULL	Stores the customer's city.

9.2 vehicles

Column	Data type	Key / constraint	Purpose
registration_no	VARCHAR(20)	UNIQUE, NOT NULL	Vehicle registration number; duplicates are prevented.
customer_id	INT	Foreign Key, NOT NULL	References customers.customer_id to record the owner.

9.3 service_jobs

Column	Data type	Key / constraint	Purpose
vehicle_id	INT	Foreign Key, NOT NULL	References vehicles.vehicle_id.
service_description	VARCHAR(100)	NOT NULL	Describes the work performed, such as Oil Change.

9.4 Why the information is separated

Separating customers, vehicles, and service jobs reduces repeated data. A customer's name and city are stored once in customers instead of being repeated in every service row. A vehicle is stored once in vehicles, while each visit creates a separate service_jobs record. This makes the structure easier to maintain and supports the one-to-many relationships in the ER model.

10. SQL Implementation

The following script creates the three tables and inserts illustrative sample records. It uses basic SQL concepts and is designed for PostgreSQL. If tables already exist and contain data you need, do not run the DROP TABLE statements; they permanently remove those tables and their stored records.

```
-- VEHICLE SERVICE CENTRE MANAGEMENT DATABASE
-- PostgreSQL SQL script
-- WARNING: The DROP statements below remove the existing tables and data.
-- Run them only when you intentionally want to recreate a clean database.

DROP TABLE IF EXISTS service_jobs;
DROP TABLE IF EXISTS vehicles;
DROP TABLE IF EXISTS customers;

-- 1. Create customers table
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL
);

-- 2. Create vehicles table
CREATE TABLE vehicles (
    vehicle_id SERIAL PRIMARY KEY,
    registration_no VARCHAR(20) UNIQUE NOT NULL,
    vehicle_model VARCHAR(50) NOT NULL,
    customer_id INT NOT NULL,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);

-- 3. Create service_jobs table
CREATE TABLE service_jobs (
    service_id SERIAL PRIMARY KEY,
    vehicle_id INT NOT NULL,
    service_date DATE NOT NULL,
    service_description VARCHAR(100) NOT NULL,
    service_cost DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (vehicle_id)
        REFERENCES vehicles(vehicle_id)
);

-- 4. Insert customers
INSERT INTO customers (customer_name, city)
VALUES
('More', 'Pune'),
('Singh', 'Mumbai'),
('Shah', 'Navi Mumbai');

-- 5. Insert vehicles
INSERT INTO vehicles (registration_no, vehicle_model, customer_id)
VALUES
('MH12AB1234', 'Hyundai i20', 1),
('MH01CD5678', 'Maruti Baleno', 2),
('MH46XY1111', 'Tata Nexon', 1),
('MH43PQ2222', 'Honda City', 3);

-- 6. Insert service records
INSERT INTO service_jobs
(vehicle_id, service_date, service_description, service_cost)
VALUES
(1, '2026-09-01', 'Oil Change', 1500),
(1, '2026-09-10', 'Brake Service', 3500),
(1, '2026-09-20', 'General Service', 2200),
(2, '2026-09-05', 'Oil Change', 1500),
(2, '2026-09-15', 'Brake Service', 3500),
(3, '2026-09-08', 'General Service', 2200),
(3, '2026-09-18', 'Brake Service', 1800),
(4, '2026-09-12', 'General Service', 2200);

-- 7. Check the data
SELECT * FROM customers;
SELECT * FROM vehicles;
SELECT * FROM service_jobs;
```

```
 1: Display vehicle owners with service history
SELECT
    customers.customer_name,
    vehicles.registration_no,
    vehicles.vehicle_model,
    service_jobs.service_date,
    service_jobs.service_description,
    service_jobs.service_cost
FROM customers
JOIN vehicles
    ON customers.customer_id = vehicles.customer_id
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id;

 2: Calculate total service revenue
SELECT SUM(service_cost) AS total_service_revenue
FROM service_jobs;

 3: Average service cost by vehicle
SELECT
    vehicle_id,
    AVG(service_cost) AS average_service_cost
FROM service_jobs
GROUP BY vehicle_id;

 4: Customers spending more than 5000
SELECT
    customers.customer_id,
    customers.customer_name,
    SUM(service_jobs.service_cost) AS total_spending
FROM customers
JOIN vehicles
    ON customers.customer_id = vehicles.customer_id
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id
GROUP BY customers.customer_id, customers.customer_name
HAVING SUM(service_jobs.service_cost) > 5000;

 5: Vehicles with more than two service records
SELECT
    vehicles.registration_no,
    vehicles.vehicle_model,
    COUNT(service_jobs.service_id) AS service_records
FROM vehicles
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id
GROUP BY
    vehicles.vehicle_id,
    vehicles.registration_no,
    vehicles.vehicle_model
HAVING COUNT(service_jobs.service_id) > 2;

-- EXTRA: Display vehicles with their owners
SELECT
    customers.customer_name,
    vehicles.registration_no,
    vehicles.vehicle_model
FROM customers
JOIN vehicles
    ON customers.customer_id = vehicles.customer_id;

-- EXTRA: Display service history for Hyundai i20
SELECT
    vehicles.registration_no,
    vehicles.vehicle_model,
    service_jobs.service_description,
    service_jobs.service_cost
FROM vehicles
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id
WHERE vehicles.vehicle_model = 'Hyundai i20';

-- EXTRA: Highest and lowest service costs
SELECT MAX(service_cost) AS highest_service_cost
FROM service_jobs;

SELECT MIN(service_cost) AS lowest_service_cost
FROM service_jobs;

-- EXTRA: Count all service records
```

```
SELECT COUNT(service_id) AS total_service_records
FROM service_jobs;
```

11. Explanation of Required Queries

Query 1 — Display vehicle owners with service history

Purpose: Show who owns each vehicle and list its recorded service details in one result. The query selects the customer name, registration number, model, service date, description, and cost. The first JOIN matches customers with vehicles through customer_id. The second JOIN matches vehicles with service_jobs through vehicle_id. Because this is an inner join, only matching linked records are returned.

Main concepts: SELECT, JOIN, primary keys, and foreign keys. This query demonstrates how data stored in separate tables can be presented as one useful report.

Query 2 — Calculate total service revenue

Purpose: Add every service_cost value in service_jobs. SUM(service_cost) returns one total for all service records. With the eight sample records in the script, the total is ■18,400. This is the total of recorded service costs in the sample database; it is not necessarily the real revenue of an actual service centre.

Main concept: SUM(), an aggregate function that calculates a total.

Query 3 — Average service cost by vehicle

Purpose: Calculate the average service cost separately for each vehicle_id. AVG(service_cost) calculates the average, while GROUP BY vehicle_id divides the rows into vehicle groups. The output contains one row per vehicle that has service records.

Main concepts: AVG() and GROUP BY. GROUP BY is not an aggregate function; it tells SQL which records should be grouped together before the aggregate calculation.

Query 4 — Customers spending more than ■5,000

Purpose: Calculate the total recorded service cost associated with each customer and show only customers whose total is greater than 5,000. The query joins the three tables so that each service cost can be associated with the correct customer. GROUP BY creates one group per customer, SUM calculates spending, and HAVING filters the grouped results.

Main concepts: JOIN, SUM(), GROUP BY, and HAVING. WHERE is generally used to filter individual rows before grouping, while HAVING filters groups after aggregation. With the sample records, More has ■11,200 in service costs and appears in the result.

Query 5 — Vehicles with more than two service records

Purpose: Count how many service records belong to each vehicle and return vehicles whose count is greater than two. COUNT(service_jobs.service_id) counts the service IDs, GROUP BY creates one group per vehicle, and HAVING keeps only groups with a count above two.

Main concepts: COUNT(), GROUP BY, HAVING, and JOIN. With the sample records, registration MH12AB1234 has three service records and qualifies for the result.

Additional queries

The script also includes basic examples to display vehicles with owners, show service information for the Hyundai i20, find the maximum and minimum service cost, and count all service records. These are optional supporting queries rather than replacements for the five main case-study queries.

12. Sample Data and Expected Results

The script inserts three customers, four vehicles, and eight service-job records. The data is illustrative and intended to make the required queries demonstrate meaningful outputs. If the records are changed, the query results will change accordingly.

Table	Sample row count	Purpose
vehicles	4	Stores vehicle information and its owner reference.

Expected result highlights

• Total service revenue: ■18,400 for the sample service records.
• Customer More's total recorded service spending: ■11,200, so More appears in the greater-than-■5,000 query.
• Vehicle MH12AB1234 (Hyundai i20) has three service records, so it appears in the more-than-two-services query.
• Average cost by vehicle is calculated only from the service records associated with each vehicle.

These are expected results for the supplied sample script. The final values in a live database should be checked by executing the queries in PostgreSQL.

13. Testing and Validation

The project can be tested by running the statements in sequence and checking that the tables, constraints, sample records, and query results behave as expected.

Test 1 — Table creation: Confirm that customers, vehicles, and service_jobs are created successfully. If the tables already exist, CREATE TABLE may report that the relation already exists. In that case, either use the existing tables or intentionally recreate them after backing up any data you need.

Test 2 — Primary key: Try to insert a duplicate primary-key value manually. PostgreSQL should reject the duplicate. SERIAL IDs are normally generated automatically when the ID is omitted.

Test 3 — Foreign key: Try to insert a vehicle with a customer_id that does not exist. PostgreSQL should reject the record because the foreign key references no customer. Similarly, a service job cannot reference a nonexistent vehicle.

Test 4 — Unique registration number: Try to insert a second vehicle with an already-used registration_no. The UNIQUE constraint should reject it.

Test 5 — Aggregate outputs: Run the revenue query and verify that the sample total is ■18,400. Check that the customer-spending query returns More and that the service-count query returns MH12AB1234 for the provided sample data.

Test 6 — Join output: Verify that the owner name and service record shown for a vehicle correspond to the customer_id and vehicle_id links in the tables.

14. Advantages and Limitations

Advantages

• Organized storage: Customer, vehicle, and service data are stored in clearly separated tables.
• Relationships: Foreign keys connect records and help preserve valid references.
• Reduced repetition: Customer details are not repeated for every service visit.
• Easy retrieval: JOIN queries combine the related information when needed.
• Basic analysis: Aggregate functions provide revenue, averages, counts, and cost ranges.
• Understandable design: The project uses a small number of tables and basic SQL, making it suitable for learning and viva demonstration.

Limitations

• The project uses sample data rather than a full real-world operational dataset.
• It does not include customer contact details, employee or mechanic records, parts inventory, invoice numbers, or payment tracking.
• It is a database implementation, not a complete user-facing web or desktop application.
• It does not currently include role-based access, automated backups, or a reporting dashboard.
• The sample data does not represent actual business revenue.

15. Future Enhancements

Possible improvements include adding customer phone and email fields, a service-category table, mechanic or employee details, spare-parts inventory, invoices, and payment records. A service status such as Pending, In Progress, or Completed could be added to service jobs. The project could also be connected to a simple application interface so staff can register customers, add vehicles, record service visits, and view reports without writing SQL manually.

Other future improvements include validation rules for costs, better handling of updates and deletions, regular database backups, and additional reports such as monthly revenue or service history for a selected registration number. These are extensions and are not required for the current basic implementation.

16. Conclusion

The Vehicle Service Centre Management Database demonstrates how a real-world service-centre problem can be represented using a relational database. The system separates customers, vehicles, and service jobs into three tables and connects them through primary and foreign keys. This structure makes the records easier to maintain and enables the service history of a vehicle to be traced back to its owner.

The SQL queries fulfill the main analytical requirements: displaying vehicle owners with service history, calculating total revenue, calculating average service cost per vehicle, finding customers whose recorded spending exceeds ■5,000, and identifying vehicles with more than two service records. The project demonstrates practical use of JOIN, SUM, AVG, COUNT, GROUP BY, and HAVING in PostgreSQL.

Overall, this project provides a clear foundation for understanding relational database design and basic SQL analysis. It can be expanded with additional tables and application features as future work.

17. Viva-Voce Questions and Answers

1. What is DBMS?

A Database Management System is software used to create, store, retrieve, update, and manage data in a database.

2. What is PostgreSQL?

PostgreSQL is a relational database management system that supports SQL and relational constraints.

3. What is SQL?

SQL stands for Structured Query Language. It is used to define, manipulate, and retrieve data in a relational database.

4. What is a table?

A table stores related data in rows and columns.

5. What is a row?

A row represents one complete record in a table.

6. What is a column?

A column represents one attribute, such as customer_name or service_cost.

7. What is a primary key?

A primary key uniquely identifies every record in a table. It cannot contain duplicate or NULL values.

8. What is a foreign key?

A foreign key refers to a key in another table and creates a relationship between the tables.

9. Which primary keys are used?

customer_id in customers, vehicle_id in vehicles, and service_id in service_jobs.

10. Which foreign keys are used?

vehicles.customer_id references customers.customer_id, and service_jobs.vehicle_id references vehicles.vehicle_id.

11. What is a one-to-many relationship?

One record in a parent table can be linked to multiple records in a child table.

12. Why are there three tables?

They separate customer details, vehicle details, and service records, reducing unnecessary repetition.

13. What does JOIN do?

JOIN combines related rows from different tables using a matching condition.

14. What does SUM() do?

SUM() adds numeric values. In this project it calculates total service revenue and customer spending.

15. What does AVG() do?

AVG() calculates the average of numeric values, used here for average service cost by vehicle.

16. What does COUNT() do?

COUNT() counts records, used here to count service jobs per vehicle.

17. What is GROUP BY?

GROUP BY puts rows with matching values into groups so aggregate functions can calculate results per group.

18. What is HAVING?

HAVING filters groups based on a condition, often involving an aggregate function.

19. Difference between WHERE and HAVING?

WHERE filters rows before grouping; HAVING filters grouped results after aggregation.

20. What is SERIAL?

In PostgreSQL, SERIAL is a convenient integer column type that generates sequential values automatically using a sequence.

21. What is NOT NULL?

It prevents a column from being left without a value.

22. What is UNIQUE?

It prevents duplicate values in the specified column or combination of columns.

23. What is referential integrity?

It means foreign-key references must point to valid records in the referenced table.

24. Why is service_cost DECIMAL?

DECIMAL stores exact numeric values with a defined number of decimal places, making it suitable for monetary amounts.

25. What does SELECT * mean?

It retrieves all columns from the specified table.

26. What is the purpose of the first query?

It displays customer names, vehicle details, and service history by joining the three tables.

27. What is the purpose of the revenue query?

It adds all service_cost values to calculate total recorded service revenue.

28. Why is HAVING used in the customer-spending query?

The condition is applied to the total spending calculated for each customer.

29. Why is HAVING used in the service-count query?

It returns only vehicles whose grouped service-record count is greater than two.

30. What could be added in the future?

Customer contact details, mechanics, parts inventory, invoices, payments, service status, and a simple application interface.

Appendix A: Complete SQL Script

This appendix repeats the complete executable SQL in one place for copying into a .sql file. Review the warning before running the DROP statements.

```
-- VEHICLE SERVICE CENTRE MANAGEMENT DATABASE
-- PostgreSQL SQL script
-- WARNING: The DROP statements below remove the existing tables and data.
-- Run them only when you intentionally want to recreate a clean database.

DROP TABLE IF EXISTS service_jobs;
DROP TABLE IF EXISTS vehicles;
DROP TABLE IF EXISTS customers;

-- 1. Create customers table
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL
);

-- 2. Create vehicles table
CREATE TABLE vehicles (
    vehicle_id SERIAL PRIMARY KEY,
    registration_no VARCHAR(20) UNIQUE NOT NULL,
    vehicle_model VARCHAR(50) NOT NULL,
    customer_id INT NOT NULL,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);

-- 3. Create service_jobs table
CREATE TABLE service_jobs (
    service_id SERIAL PRIMARY KEY,
    vehicle_id INT NOT NULL,
    service_date DATE NOT NULL,
    service_description VARCHAR(100) NOT NULL,
    service_cost DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (vehicle_id)

        REFERENCES vehicles(vehicle_id)
);

-- 4. Insert customers
INSERT INTO customers (customer_name, city)
VALUES
('More', 'Pune'),
('Singh', 'Mumbai'),
('Shah', 'Navi Mumbai');

-- 5. Insert vehicles
INSERT INTO vehicles (registration_no, vehicle_model, customer_id)
VALUES
('MH12AB1234', 'Hyundai i20', 1),
('MH01CD5678', 'Maruti Baleno', 2),
('MH46XY1111', 'Tata Nexon', 1),
('MH43PQ2222', 'Honda City', 3);

-- 6. Insert service records
INSERT INTO service_jobs
(vehicle_id, service_date, service_description, service_cost)
VALUES
(1, '2026-09-01', 'Oil Change', 1500),
(1, '2026-09-10', 'Brake Service', 3500),
(1, '2026-09-20', 'General Service', 2200),
(2, '2026-09-05', 'Oil Change', 1500),
(2, '2026-09-15', 'Brake Service', 3500),
(3, '2026-09-08', 'General Service', 2200),
(3, '2026-09-18', 'Brake Service', 1800),
(4, '2026-09-12', 'General Service', 2200);

-- 7. Check the data
SELECT * FROM customers;
SELECT * FROM vehicles;

SELECT * FROM service_jobs;
```

```
-- QUERY 1: Display vehicle owners with service history
SELECT
    customers.customer_name,
    vehicles.registration_no,
    vehicles.vehicle_model,
    service_jobs.service_date,
    service_jobs.service_description,
    service_jobs.service_cost
FROM customers
JOIN vehicles
    ON customers.customer_id = vehicles.customer_id
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id;

-- QUERY 2: Calculate total service revenue
SELECT SUM(service_cost) AS total_service_revenue
FROM service_jobs;

-- QUERY 3: Average service cost by vehicle
SELECT
    vehicle_id,
    AVG(service_cost) AS average_service_cost
FROM service_jobs
GROUP BY vehicle_id;

-- QUERY 4: Customers spending more than 5000
SELECT
    customers.customer_id,
    customers.customer_name,
    SUM(service_jobs.service_cost) AS total_spending
FROM customers
JOIN vehicles

    ON customers.customer_id = vehicles.customer_id
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id
GROUP BY customers.customer_id, customers.customer_name
HAVING SUM(service_jobs.service_cost) > 5000;

-- QUERY 5: Vehicles with more than two service records
SELECT
    vehicles.registration_no,
    vehicles.vehicle_model,
    COUNT(service_jobs.service_id) AS service_records
FROM vehicles
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id
GROUP BY
    vehicles.vehicle_id,
    vehicles.registration_no,
    vehicles.vehicle_model
HAVING COUNT(service_jobs.service_id) > 2;

-- EXTRA: Display vehicles with their owners
SELECT
    customers.customer_name,
    vehicles.registration_no,
    vehicles.vehicle_model
FROM customers
JOIN vehicles
    ON customers.customer_id = vehicles.customer_id;

-- EXTRA: Display service history for Hyundai i20
SELECT
    vehicles.registration_no,
    vehicles.vehicle_model,
    service_jobs.service_description,

    service_jobs.service_cost
FROM vehicles
JOIN service_jobs
    ON vehicles.vehicle_id = service_jobs.vehicle_id
WHERE vehicles.vehicle_model = 'Hyundai i20';

-- EXTRA: Highest and lowest service costs
SELECT MAX(service_cost) AS highest_service_cost
FROM service_jobs;

SELECT MIN(service_cost) AS lowest_service_cost
```

```
FROM service_jobs;

-- EXTRA: Count all service records
SELECT COUNT(service_id) AS total_service_records
FROM service_jobs;
```